

IMPLEMENTED SYSTEM REFERENCE

VGGTBEV Method II

Full Training and Runtime Pipeline — Learned VGGT-Normalized Range, Dual Observed-Area BEV Outputs

FROZEN VGGT-OMEGA

LIVE PREDICTED GEOMETRY

512 / 800 OUTPUTS

NO CAMERA HEIGHT

Training products

Single-frame masked BEV and cumulative merged masked BEV

Runtime input

Contiguous RGB history only; no GT calibration, pose, trajectory, or BEV

Spatial convention

Per-sequence VGGT reference normalization + learned positive zoom

Reference maps

End-to-end training/runtime block maps + tensor-level I/O maps

Implementation

/home/wolfie/Project/VGGTBEV

Prepared from the current codebase, active P2 configuration, validated 20,000-session manifest, and *Rigorous Formulation of Three BEV Prediction Methods for VGGT- Ω -Based Multi-View Reconstruction*.
Document date: 28 July 2026

1. Purpose and system boundary

This document describes what the current VGGTBEV project actually trains and runs. It is a Method II geometry-aware lift-and-splat system: VGGT- Ω extracts features and predicts geometry; a standalone trainable BEV module projects that evidence into two grids and predicts observed occupancy. VGGT itself is not modified.

20,000 selected complete simulator sessions	225,646 aligned train + validation samples	512 / 800 single / merged output side lengths
---	--	---

Scope

Included now ✓ RGB-to-VGGT live feature and geometry inference ✓ VGGT-predicted depth, confidence, intrinsics, and relative poses ✓ Robust ground orientation and origin alignment ✓ VGGT-relative normalization with learned per-sequence range ✓ Single-frame and cumulative observed-area BEV heads ✓ Explicit surface and free-space evidence	Intentionally excluded • Revealed or unobserved-area completion • GT depth, GT intrinsics, GT extrinsics, or GT trajectory as model input • Camera-height scale recovery • A learned standalone scale predictor • Fine-tuning VGGT-Ω • Method I direct token-to-BEV and Method III residual fusion
---	--

Output contract Each head predicts an observation logit and an occupancy logit. The final grayscale BEV is occupied = 0, unknown = 112, free = 255. Occupancy is supervised only where the target says the cell was observed.

Document map

2. Method II alignment and deliberate deviations 3. Training data and leakage boundary 4. Training block map and tensor I/O map 5. RGB preprocessing and batching 6. Frozen live VGGT extraction 7. Geometry reconstruction and raw scale 8. Feature lift, splat, ray evidence, decoder 9. Dual targets, losses, and metrics	10. Optimization, DDP, logging, checkpoints 11. Runtime block map and tensor I/O map 12. Offline and live-server runtime contracts 13. Saved artifacts and comparison outputs 14. Invariants, failure modes, and caveats 15. Validation and launch procedure 16. Active configuration reference
--	--

2. Alignment with Method II in the source PDF

Section 4 of the source PDF defines Method II as feature lifting using predicted depth, intrinsics, and extrinsics, followed by confidence-weighted BEV splatting and a compact 2D decoder. The implementation follows that architecture directly.

Method II requirement	Current implementation	Status
Image feature attached to a 3D point	Final VGGT patch token + sampled depth at each 16-pixel patch center	Aligned
Predicted intrinsics and extrinsics	VGGT camera head; no simulator camera tensor enters the model	Aligned
Ground-aligned coordinates	All points enter the latest-camera frame; robust plane fit defines right/forward/up axes	Aligned
Confidence filtering and weighted splat	VGGT confidence is converted to [0,1], thresholded, and used as bilinear-splat weight	Aligned
Free versus unknown reasoning	Explicit ray evidence stops before the predicted surface; cells without evidence can remain unknown	Aligned
2D BEV decoder	Point MLP, evidence raster, convolutional stem, two residual blocks, two output logits	Aligned
Metric scale	No metric cue. VGGT depth and translation share VGGT’s implicit reconstruction scale.	Deliberately non-metric
Unobserved completion	Forbidden in current targets and loss	Deferred

Scale interpretation

$$X_{\text{camera}} = d \, K^{-1} \, p \quad \text{and} \quad X_{\text{reference}} = T(\text{VGGT poses}) \cdot X_{\text{camera}}$$

VGGT does not produce a separate scale scalar. Its predicted depths and camera translations already live in one shared reconstruction coordinate system. The active mode `vggt_normalized` first preserves the common raw reconstruction, then divides all aligned geometry by a per-sequence VGGT reference radius. An integrated trainable normalizer predicts one positive normalized-units-per-pixel value for that sequence.

What “6.5” and “10” mean in the active normalized mode

They select the existing simulator supervision images only. Their extent tensors are removed before the model forward and never define runtime grids. The model sees a 512-pixel single target and an 800-pixel nearest-resized merged target, so the source extents remain implicit properties of the training distribution rather than hard output limits.

Why this is still Method II

Spatial support is still determined by VGGT geometry before decoding; the trainable module does not directly query arbitrary BEV cells from tokens. Therefore this remains geometry-aware lift-and-splat, not Method I. The absence of metric calibration changes scale semantics, not the architectural method.

3. Training data and the leakage boundary

Authoritative collection

Dataset root	/data/disk_7t/diwen/VGGT_DATA/data_build_unlimited
Selection	database_manifest.json , 20,000 complete sessions spanning simulator scenes
Train split	16,242 sessions, 183,510 target frames
Validation split	3,758 sessions, 42,136 target frames
Split rule	Scene grouping + stable SHA-256 hash; sessions from one scene cannot leak across splits
Image input	RGB camera frames, indexed and contiguous from frame 0
Single target	bev_6p5m/masked/frame_XXXXXX.png
Merged target	bev_6p5m/merged_masked_10m/frame_XXXXXX.png , fusion version 2

What simulator information is used

Data item	Dataset/indexing	Model forward	Purpose
RGB frames	Yes	Yes	Only sensory input
Masked single BEV	Yes	Loss target only	Current-frame observed occupancy
Masked merged BEV	Yes	Loss target only	Cumulative observed occupancy
Complete/revealed BEVs	Optional audit reader	Forbidden	Never used for this P2 objective
GT trajectory JSONL	Index/alignment only	Forbidden	Find camera files, target metadata, and contiguous frame IDs
GT intrinsics/extrinsics/depth	May exist on disk	Forbidden	Collector/audit information only
Camera height	May exist on disk	Absent	Not read into a raw-mode sample

Hard enforcement

Before moving a batch to the GPU, `_move_batch` rejects complete/revealed labels, simulator intrinsics, camera-to-world matrices, floor transforms, and floor heights. In `vggt_normalized` it also rejects `camera_height_m` and all fixed extent tensors. This makes train/runtime geometry-source parity a code-level invariant rather than a convention.

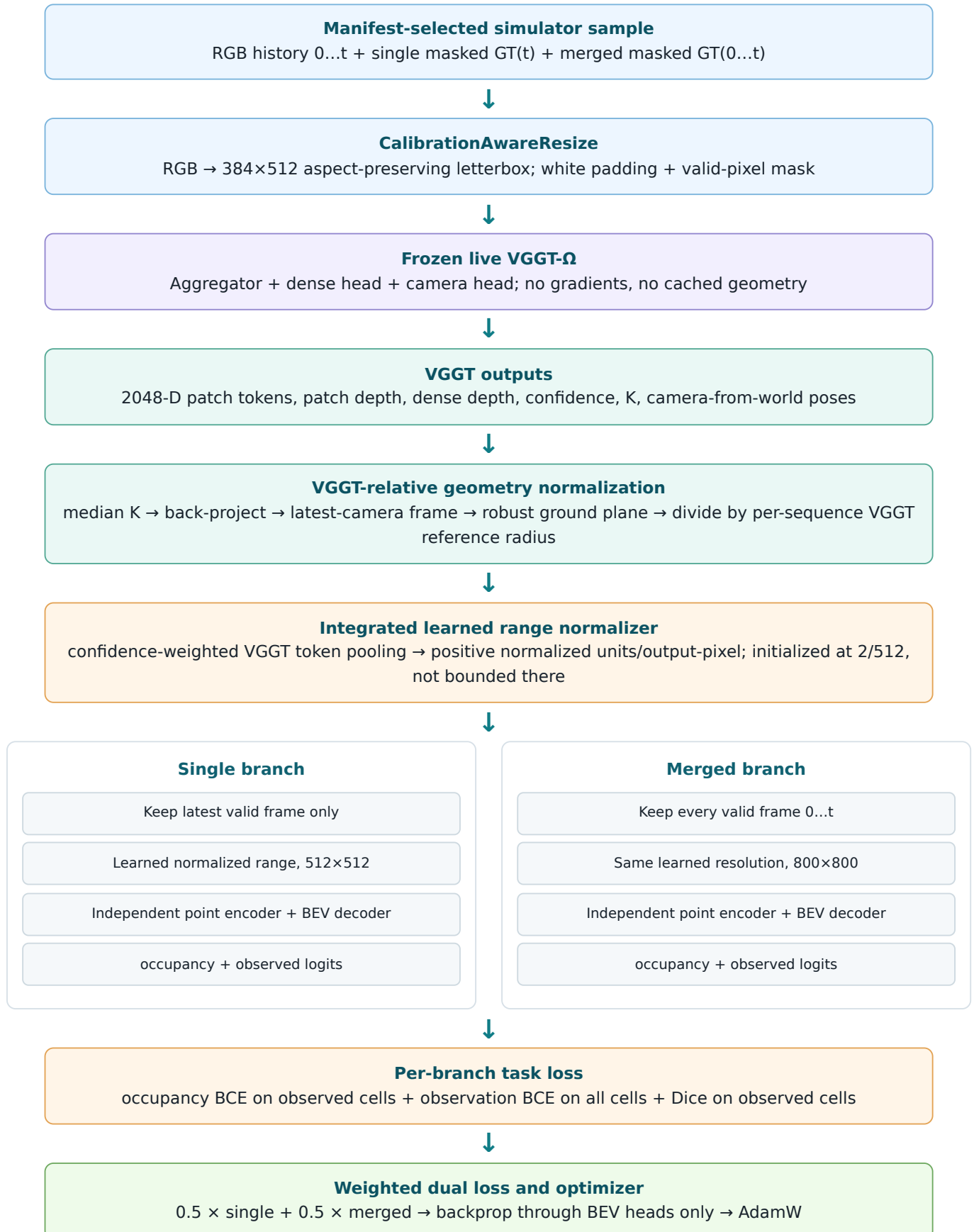
Target semantics

The ideal simulator shadowcaster establishes visibility. Version-2 merged labels accumulate visibility across frames while both masked and complete variants obtain occupancy from the same static collision truth. Unknown is not treated as free. This distinction is essential for the observation loss and for navigation-safe free-space supervision.

4. Training pipeline — block map and I/O map

BLOCK MAP

□ data/input □ frozen □ geometry □ trainable □ output/loss



VGGT executes once per batch and its extraction is shared by both branches. The single and merged heads are separate trainable modules; they do not share point-encoder or decoder weights.

TRAINING I/O MAP

Dataset batch

- `images`: [B,N,3,384,512]
- `image_valid`: [B,N,384,512]
- `frame_valid`: [B,N]
- `single target`: [B,512,512]
- `merged target`: [B,800,800]



Model interface

- Frozen VGGT tokens: [B,N,768,2048]
- VGGT depth/confidence: [B,N,768]
- VGGT K: [B,N,3,3]
- VGGT camera-from-world: [B,N,4,4]
- Learned units/output-pixel: [B]



Trainable outputs

- Single occupancy logit + observed logit: each [B,512,512]
- Merged occupancy logit + observed logit: each [B,800,800]
- Single + merged losses → scalar total loss
- Checkpoint: normalizer + both BEV heads + optimizer state

Extent boundary

The 6.5 m single crop and 10 m merged crop identify existing label files and remain in metadata for auditing. No 6.5/10 extent tensor, camera height, simulator pose, simulator K, or simulator depth enters the model interface. No target regeneration is required.

5. RGB preprocessing, sample construction, and batching

Per-frame preprocessing

- 1. Open the simulator PNG and convert to RGB.
- 2. Scale it uniformly to fit inside 384×512 without changing aspect ratio.
- 3. Center the resized image on a white 384×512 canvas.
- 4. Create a Boolean valid-pixel mask that excludes the padded margins.
- 5. Normalize RGB values to floating point [0,1].

Both dimensions are divisible by the VGGT patch size 16. Each frame therefore produces a 24×32 patch lattice, or **768 patch tokens**.

History rules

Product	RGB history supplied	Target	Reason
Single	Only frame t when trained alone; full history in dual mode	Single masked BEV at t	Dual mode reuses the shared temporal extraction
Merged	Every frame 0...t	Merged masked BEV at t	The label is cumulative from frame zero

A truncated sliding window is not an equivalent merged input because its evidence history no longer matches the cumulative target. Runtime follows the same contiguous 0...t rule.

Collation

Histories have different lengths, so the collator pads to the longest N in the batch and emits: `images[B,N,3,384,512]` , `image_valid[B,N,384,512]` , and `frame_valid[B,N]` . Targets are stacked separately for the two branches.

Critical current limitation: VGGT sees padded frames
`frame_valid` is applied after VGGT extraction, but the VGGT temporal aggregator itself has no padding-mask input. With variable history lengths and batch size greater than one, white padded frames can influence valid tokens. The active config says batch size 6/GPU, so a long run is safe only after adding equal-length bucketing, or by overriding batch size to 1. Merely keeping `frame_valid` does not solve this upstream attention contamination.

Safe options

- 1. **Immediate:** train with `--batch-size-per-gpu 1` .
- 2. **Preferred scale-up fix:** batch only samples with the same history length N.
- 3. **Alternative architecture work:** add a tested attention padding mask inside VGGT, which would modify the base-model integration and requires new validation.

6. Frozen live VGGT-Ω extraction

Backbone loading

VGGT source	/data/disk_14t/diwen/vggt
Checkpoint	checkpoints/VGGT-Omega-1B-512/model.pt
Checkpoint loading	Memory-mapped CPU load, strict state-dict match, then move to assigned device
Gradient state	All parameters <code>requires_grad=False</code> ; model held in eval mode
Precision	bfloat16 autocast on supported CUDA devices, otherwise float16; decoded outputs converted to float32

Outputs used by Method II

Appearance Patch features Final aggregator tokens after the special-token prefix. Shape: [B,N,768,2048]	Surface geometry Depth + confidence Dense-head predictions sampled at each patch center; dense maps retained for baseline rendering.	Camera geometry K + poses Camera-head encoding decoded into intrinsics and camera-from-world extrinsics for every frame.
--	--	--

Confidence conversion

VGGT confidence is represented as $1 + \exp(\text{logit})$. The implementation converts it exactly to a sigmoid-like $[0,1]$ confidence:

$$c = (\text{confidence}_{\text{VGGT}} - 1) / \text{confidence}_{\text{VGGT}}$$

A patch is initially valid only if its frame is valid, its patch center lies in the unpadded image region, and its depth is finite and positive.

Why extraction stays live during training

- Training sees the same VGGT error distribution as runtime.
- No accidental replacement with perfect simulator geometry is possible.
- The single and merged branches receive mutually consistent features, depth, and poses from one forward.
- It is slower than cached features, but it preserves the required train/runtime contract.

7. Predicted geometry and learned VGGT-normalized range

Step 1 — stabilize intrinsics

VGGT may predict slightly different focal lengths and principal points across frames. For each sequence, the implementation takes the median valid-frame values of f_x , f_y , c_x , and c_y and constructs one fixed K used for every frame. Invalid or non-positive focal estimates fail fast.

Step 2 — back-project patch centers

$$X_i(p) = d_i(p) K^{-1} [u, v, 1]^T$$

The 768 sampled depths per frame become 3D points in that frame's OpenCV camera coordinates. VGGT camera poses then transform all points and camera origins into the **latest valid camera frame**, which is the local reference for both outputs.

Step 3 — robust ground fit

- Candidate below-camera points are selected between the 0.55 and 0.995 y-quantiles.
- A confidence-weighted plane is fitted with five IRLS iterations and Huber delta 2.5.
- The normal must point toward camera-up and remain within 35° tilt.
- At least 48 points are preferred; sparse or invalid cases use a level fallback plane.
- Logs report inlier fraction, fallback fraction, tilt, and predicted height in raw VGGT units.

Step 4 — define the BEV basis

The floor-plane origin is translated to zero. Camera-right is projected onto the plane and normalized; forward is the cross product of up and right. Points become (right, forward, height) in this ground-aligned frame.

Step 5 — normalize and learn the raster zoom

$$X_{\text{norm}} = X_{\text{aligned}} / \text{mean}_{\text{valid}} ||(\text{right}, \text{forward})|| \quad \text{and} \quad \text{units/pixel} = f_{\theta}(\text{VGGT tokens})$$

Camera height is neither read nor estimated as a conversion target. The common VGGT geometry is normalized by its own per-sequence reference radius. A small integrated MLP predicts a positive zoom with no configured upper bound. The 512 and 800 grids share this learned units-per-pixel value, so the merged grid spans 800/512 times the normalized width.

Ground alignment and reference normalization do **not** make coordinates metric. Physical coverage remains free to change between sequences; the output is defined by normalized pixel coordinates and the learned zoom.

8. Feature lift, splat, free-space evidence, and BEV decoder

Point feature encoder

Each lifted point combines its 2048-D appearance token with three geometric descriptors:

$$f_{\text{point}} = \text{MLP}([f_{\text{VGGT}}, \text{height}, \text{radial_distance}, \log(\text{confidence})]) \in \mathbb{R}^{96}$$

The encoder is LayerNorm(2051) → Linear(2051,96) → GELU → Linear(96,96). In normalized mode, the configured metric height band is disabled because its thresholds have no valid physical interpretation.

Validity gates

- Frame and image location must be valid.
- Depth must be finite and between 0.0001 and 10,000 in raw VGGT units.
- Converted confidence must be at least 0.05.
- Projected point coordinates must be finite and fall inside the selected grid.

Confidence-weighted bilinear splat

Each point distributes its encoded feature and confidence to the four neighboring BEV cells. Per-cell features are the weighted sum divided by accumulated confidence. This preserves smooth sub-cell placement and produces a surface-evidence channel.

Branch	Frames used	Numeric width	Raster	Raw units/pixel
Single	Latest valid frame	Learned per sequence	512×512	Learned shared value
Merged	All valid frames 0...t	Learned per sequence	800×800	Same learned value

Free-space ray evidence

For every valid point, a ground-plane ray is sampled from its camera origin toward the surface. The active config uses 768 samples. Sampling stops one grid cell before the endpoint and clips at the square boundary, so surface endpoints are not carved as free. The accumulated ray evidence is transformed with log(1+x).

2D decoder

The decoder input has 98 channels: 96 splatted features, log surface evidence, and free evidence. A 3×3 convolutional stem and two residual blocks refine the grid; a final 1×1 convolution produces occupancy and observation logits. The two branches have identical architecture but independent parameters.

9. Dual supervision, loss, rendering, and validation metrics

Label decomposition

PNG value	Meaning	Observed target	Occupied target
0	Occupied and observed	1	1
112	Unknown / not established by visibility	0	Not supervised
255	Free and observed	1	0

Per-branch loss

$$L_{\text{task}} = 1.0 L_{\text{occupancy BCE} \mid \text{observed}} + 0.5 L_{\text{observation BCE}} + 0.25 L_{\text{occupancy Dice} \mid \text{observed}}$$

The observation loss covers all cells. Occupancy BCE and Dice are evaluated only on observed target cells; unknown pixels cannot leak an implicit free label into training. If a target has no observed cells, the occupancy and Dice terms are exactly zero.

Dual-task combination

$$L_{\text{total}} = 0.5 L_{\text{single}} + 0.5 L_{\text{merged}}$$

Both weights must be positive. The same label semantics and component weights are used for both products.

Metrics

- **Observed IoU:** thresholded observation prediction versus observed target mask.
- **Occupied IoU on observed:** occupancy prediction evaluated only where the target is observed.
- Metrics are reported independently for single and merged branches.

Runtime rendering

With threshold τ (default 0.5), a cell is unknown if observation probability is below τ . Otherwise it is occupied if occupancy probability is at least τ , and free if it is below τ .

These outputs are **masked observed-area BEVs**. The decoder may smooth or denoise evidence, but the task does not supervise the occupancy of unobserved cells. A completion result would require a different target and explicitly separated evaluation.

10. Optimization, distributed training, logging, and checkpoints

Active optimizer configuration

Epochs	10
Configured batch per GPU	6 — see the variable-history warning in Section 5
Optimizer	AdamW
Learning rate	2×10^{-4}
Weight decay	1×10^{-4}
Gradient clipping	Global norm 5.0
Scheduler	None in the current implementation
Log interval	Every 10 optimizer steps, plus step 1
Checkpoint interval	Every 500 steps and after each epoch

Distributed execution

One process runs per GPU. Each rank independently loads the same frozen VGGT checkpoint. Frozen VGGT parameters and buffers are excluded from DDP synchronization, avoiding a redundant multi-gigabyte broadcast. DDP synchronizes only trainable single/merged head parameters. Backend selection is `nccl` by default or `gloo` through `VGGTBEV_DISTRIBUTED_BACKEND`.

Logged telemetry

Learning total/component losses, world size, batch size, effective batch size, epoch, global step, mean history length	Geometry health VGGT reference radius, learned normalized units/pixel, learned single/merged spans, ground inliers, fallback fraction, and tilt
--	---

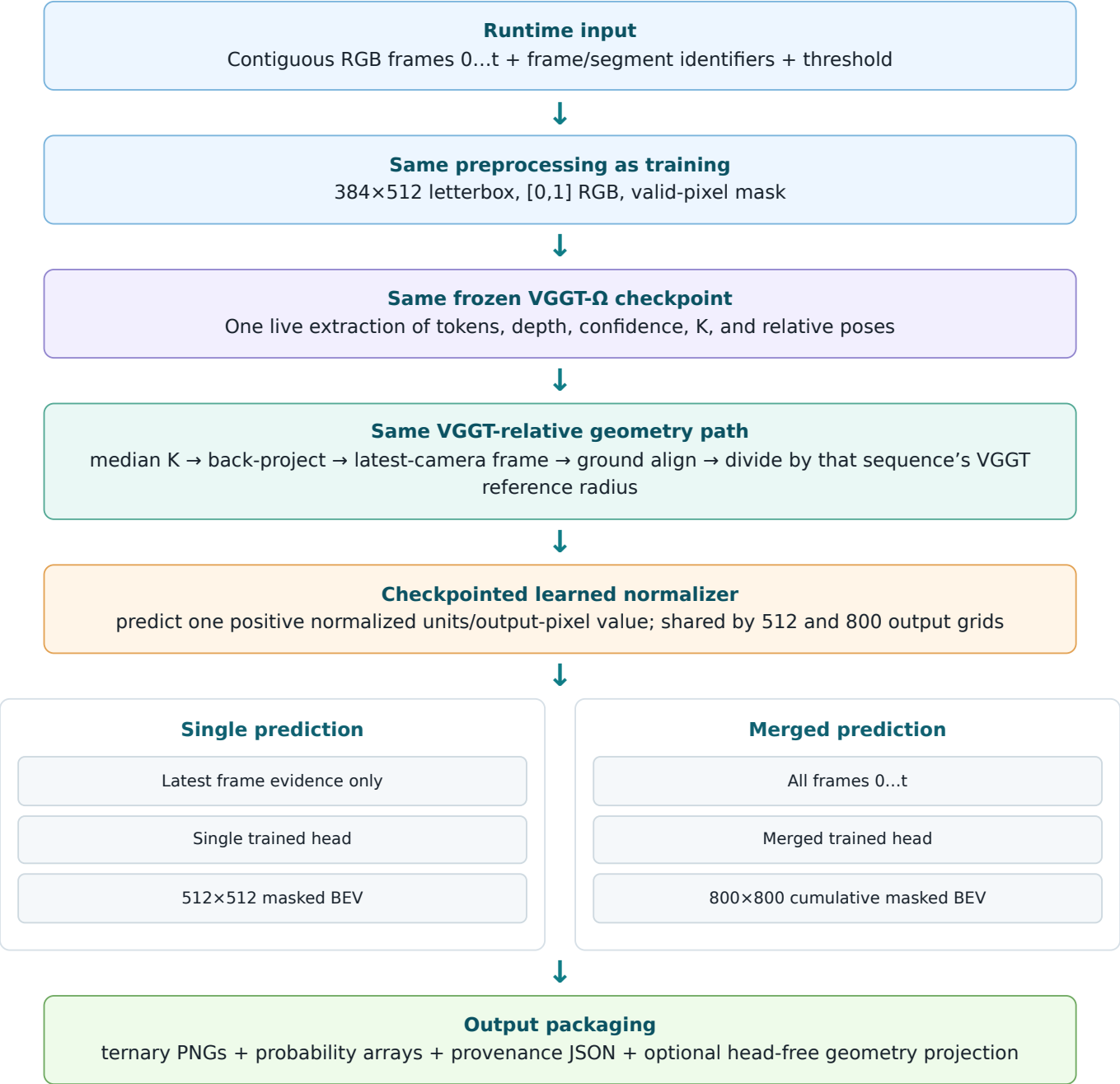
Checkpoint content

- Epoch, completion flag, and global step
- Single head state
- Merged head state
- Fixed compatibility `log_depth_scale` tensor
- AdamW optimizer state
- Complete configuration snapshot

The frozen 4.4 GB VGGT checkpoint is not duplicated. Runtime must have access to both the compact Method II checkpoint and the original VGGT-Ω checkpoint.

11. Runtime pipeline — block map and I/O map

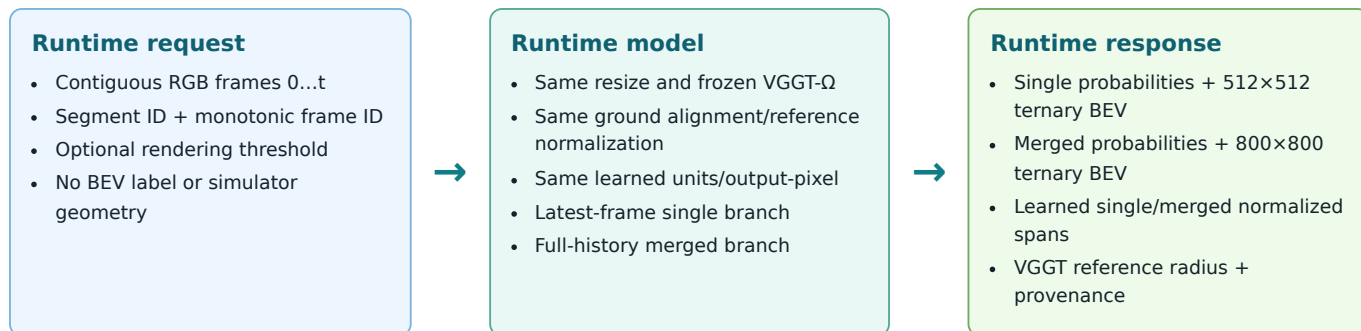
BLOCK MAP



Runtime does not receive GT

No simulator intrinsic matrix, camera pose, trajectory, depth, camera height, or BEV label is present in the raw P2 runtime batch. VGGT estimates all camera geometry from RGB.

RUNTIME I/O MAP



$$\text{merged span} / \text{single span} = 800 / 512 = 1.5625$$

This ratio is fixed because both grids share resolution in learned normalized units per pixel. The absolute normalized span varies per sequence and is not clamped to either source-label crop.

Train/runtime parity

Stage	Training	Runtime
RGB preprocessing	CalibrationAwareResize	Same class and size
VGGT execution	Live, frozen	Live, frozen
Geometry source	VGGT predictions	VGGT predictions
Range	VGGT-normalized, learned per sequence	Same learned normalizer
Ground alignment	Robust fit	Same robust fit
Targets	Masked GT for loss	None

12. Offline inference and stateful live service

Offline sequence inference

`load_vggnav_runtime_sequence` scans only `camera/frame_*.png`. Frame IDs must be contiguous from zero. It loads frames 0 through the selected target, preprocesses them, and creates a `RuntimeSequence`. In normalized mode, camera height remains `None`.

```
.venv/bin/vggt-bev-infer \  
--checkpoint runs/p2_vggt_geometry_6p5m_10m/latest.pt \  
--session /path/to/session \  
--target-frame -1 \  
--output-dir artifacts/p2_runtime \  
--device cuda
```

Stateful HTTP service

`RuntimeHistory` owns one cumulative segment. A new segment ID resets history. Within a segment, `frame_seq` must strictly increase. Requests carry a base64 PNG; the service appends the frame only after validation and rolls it back if inference fails.

Endpoint	Purpose	Main fields
<code>GET /health</code>	Readiness and model metadata	checkpoint, output sizes, learned-range mode, history count, geometry source
<code>POST /predict</code>	Append one frame and infer	segment_id, frame_seq, image_png_base64, threshold
<code>POST /reset</code>	Clear cumulative state	new segment_id

History is capped (default 34). Once full, the service returns HTTP 409 and the caller must start a new segment. The entire history is reprocessed by VGGT on each prediction; there is no cached temporal key/value state in the current implementation.

```
.venv/bin/vggt-bev-runtime-server \  
--checkpoint runs/p2_vggt_geometry_6p5m_10m/latest.pt \  
--device cuda --host 127.0.0.1 --port 8765 --max-history 34
```

Multi-checkpoint comparison mode

The runtime can load several independently trained single-output extents while sharing one frozen VGGT adapter. One extraction is reused by all heads. This is intended for the 3.5/5/6.5 legacy visualizer; the active scale-up P2 checkpoint itself uses normalized 512/800 pixel outputs.

13. Runtime outputs, provenance, and geometry baseline

Saved artifacts

Artifact	Contents
single_masked.png	Current-frame 512×512 ternary observed BEV
merged_masked.png	Cumulative 0...t 800×800 ternary observed BEV
geometry_single.png	Head-free VGGT geometry projection for the latest frame
geometry_merged.png	Head-free geometry projection from all frames
probabilities.npz	Occupancy and observation probabilities for both learned outputs
runtime.json	Checkpoint hash, step, frame history, threshold, scale mode, grid semantics, K estimate, and file paths

Head-free projection versus trained Method II

Head-free baseline

Uses VGGT dense depth, predicted K and poses, ground alignment, confidence, and deterministic rasterization. It has no learned point encoder or 2D decoder.

Trained Method II

Uses the same estimated geometry plus patch appearance, learned evidence aggregation, explicit rays, and a convolutional decoder. It can denoise systematic VGGT projection errors within the supervised observed region.

What the comparison can prove

A synchronized visualizer is useful qualitative evidence, but a few scenes cannot prove broad superiority. A defensible claim requires held-out-scene statistics for observed IoU, occupied-on-observed IoU, free-space precision/recall, navigation collision rate, path success, inference time, and robustness across camera FOV, height, assets, and VGGT scale variation.

The geometry images are a diagnostic baseline, not a training target. Simulator GT stays in the comparison process only for display/evaluation and is never sent to the runtime service.

14. Invariants, failure modes, and scientific caveats

Non-negotiable invariants

- ✓ Depth and pose translations must come from the same VGGT forward.
- ✓ Depth, pose translations, points, and camera origins must share the same VGGT reference normalization.
- ✓ Training and runtime must use the same preprocessing and VGGT checkpoint.
- ✓ Merged input history must begin at frame zero and match the cumulative label history.
- ✓ Unknown target cells must never be converted to free occupancy supervision.
- ✓ Complete/revealed BEVs must remain outside this training objective.

Main failure modes

Failure	Observable symptom	Mitigation / gate
Cross-sequence VGGT scale drift	Physical coverage and obstacle placement vary	Measure scale distribution; evaluate by scene; add a metric cue only if physical scale becomes required
Variable-length padding enters VGGT attention	Batch-dependent predictions or degraded merged learning	Batch size 1 or equal-history bucketing
Ground fit fallback is frequent	Unstable rotation/origin; distorted splats	Track fallback/inlier telemetry and inspect problematic views
Pose drift across long histories	Duplicated/thick walls in merged map	Bound history, quantify drift, consider submaps or pose refinement later
VGGT K varies frame-to-frame	Trajectory deformation and inconsistent projection	Current median-K stabilization
Class imbalance	Mostly-free output, low occupied recall	Monitor occupied IoU; tune loss only on a held-out validation protocol
Target/data misalignment	Sharp but consistently shifted prediction	Axis/origin tests and visual overlays before scale-up

Method II scale caveat

The source PDF’s shared-scale proposition requires geometry and BEV grid bounds to be scaled jointly. The current design does this in normalized coordinates using one learned units-per-pixel value for both outputs. The 6.5 m/10 m crops still shape the supervision distribution, so the model may learn ranges similar to them, but those values are not model inputs or hard runtime constraints. This remains non-metric observed-BEV prediction.

15. Validation and recommended launch procedure

Checks already completed for the current update

- ✓ Local complete unit suite: 50 tests passed.
- ✓ Remote source files verified byte-for-byte with SHA-256.
- ✓ Remote compile check passed.
- ✓ Remote non-fixture tests: 26 passed.
- ✓ Remote production data-only loader validated 20,000 sessions and 225,646 samples.
- ✓ First production sample contains RGB + 512×512/800×800 labels, no camera height, no fixed extent tensor, and no simulator GT geometry tensor.

Before a long GPU run

1. Resolve variable-history batching: override batch size to 1 or implement/test equal-length buckets.
2. Run data-only validation and save the dataset report with the run.
3. Run one full live-VGGT forward/backward step on the target GPUs.
4. Overfit a very small scene subset; verify both IoUs rise and rendered axes are correct.
5. Inspect learned range, reference-radius, and ground fallback statistics across all three simulators.
6. Start DDP and monitor GPU utilization, memory, dataloader stalls, loss, IoU, and ground telemetry.

Data-only gate

```
.venv/bin/python -m vgg_t_bev.cli_train \  
  --config configs/remote_hm3d_p2_vgg_t_geometry_6p5m_10m.toml \  
  --data-only --device cpu --num-workers 0
```

Safe one-step integration gate

```
CUDA_VISIBLE_DEVICES=0 .venv/bin/python -m vgg_t_bev.cli_train \  
  --config configs/remote_hm3d_p2_vgg_t_geometry_6p5m_10m.toml \  
  --batch-size-per-gpu 1 --max-train-steps 1 --skip-validation \  
  --output-dir runs/p2_raw_scale_smoke
```

Four-GPU launch using Gloo, after the batching gate is satisfied

```
CUDA_VISIBLE_DEVICES=0,1,2,3 \  
VGGTBV_DISTRIBUTED_BACKEND=gloo \  
.venv/bin/python -m torch.distributed.run \  
  --standalone --nproc_per_node=4 \  
  -m vgg_t_bev.cli_train \  
  --config configs/remote_hm3d_p2_vgg_t_geometry_6p5m_10m.toml \  
  --batch-size-per-gpu 1 --num-workers 4
```

16. Active P2 configuration reference

Data

extent_key	bev_6p5m
target_mode	both
geometry_source	vggt
coordinate_mode	vggt_normalized
source label crops	6.5 m single / 10 m merged; selection and audit metadata only
fusion version	2
input image	384×512
split	scene / stable_hash

Model

VGGT execution	live
patch / feature	16 / 2048
hidden width	96
BEV feature grids	256 / 400
outputs	512 / 800
ray steps	768
scale mode	vggt_normalized
global depth scale	fixed at 1; not trainable
range normalizer	trainable per sequence; positive and not upper-bounded
shared resolution	one learned normalized-units/output-pixel value

Geometry

stabilize K	true
height filter	false
min confidence	0.05
raw depth range	0.0001-10,000
ground points	48 minimum
IRLS / Huber	5 / 2.5
max tilt	35°
raw applied depth scale	1.0 before per-sequence reference normalization

Training

epochs	10
batch/GPU	6 configured; 1 safe now
learning rate	0.0002
weight decay	0.0001
loss weights	1.0 / 0.5 / 0.25
task weights	0.5 / 0.5
seed	7
checkpoint steps	500

Key code locations

Dataset and split	src/vggt_bev/data/vggnav_dataset.py
Collation	src/vggt_bev/data/collate.py
Frozen VGGT adapter	src/vggt_bev/models/vggt_adapter.py
Method II system and heads	src/vggt_bev/models/method2.py
Ground alignment	src/vggt_bev/geometry/ground.py
Splat and rays	src/vggt_bev/geometry/splat.py

Loss / metrics	src/vggt_bev/losses.py / metrics.py
Training entry point	src/vggt_bev/cli_train.py
Runtime / service	src/vggt_bev/runtime.py / runtime_server.py

Terminology note: several compatibility APIs retain names such as `extent_m`, `meters_per_pixel`, and `minimum_depth_m`. When `coordinate_mode = "vggt_normalized"`, those extent values are label-source audit metadata only and are absent from model batches.